

# 2026-04-11 AIWS × SAM3D × Cadrille 主管汇报

作者: 詹铠铭

日期: 2026-04-11

定位: 面向汇报的高层总结, 重点回答环境、全量实验规模、运行时间、显存/共享内存约束, 以及后续如何继续推理。

## 1. 本次工作的核心结论

本轮工作已经把 AIWS5.2 清洗版数据集上的单图 3D 重建流程跑通, 并把后续 CAD 生成链路验证到可复用状态:

1. **SAM3D** 环境已经稳定可复现, 并在 RXL 服务器上完成 1418 个正式样本的全量推理。
2. **SAM3D** 全量运行完成率 100%, 且具备较完整的单样本时间与显存统计。
3. **Cadrille** 的 PC 模态全量推理已跑通, 说明 SAM3D STL 输出可以稳定进入 CAD 生成链路。
4. **Cadrille** 的 IMG 模态需要额外的共享内存 (shm) 调优, 在默认配置下会因 DataLoader worker 共享内存不足失败, 但在修复后已经完成全量成功重试。
5. 后续若继续做批量推理, 建议采用“**SAM3D 与 Cadrille 解耦、PC / IMG 分开跑**”的运维策略, 而不是直接一次性混合发射。

## 2. 数据集与硬件约束

### 2.1 正式数据口径

- 数据集: aiws5.2-usable
- 正式任务总数: 1418
- 子集分布:
  - V1 : 593
  - V2 : 524
  - NEW : 301

### 2.2 数据集整理过程与目录含义

本项目的数据原始状态更接近平铺式资源池:

- aiws5.2-dataset/images/ : 所有 RGB 图像基本混放在同一个目录
- aiws5.2-dataset/depth/ : 所有深度文件混放在同一个目录

- `isat_annotations/` : ISAT 标注 JSON, 作为最终标注真值来源
- `train.json / val.json` : 只提供 train / val 归属信息

这种原始组织方式不利于批量实验, 主要问题是:

1. 不方便直接按 `V1 / V2 / NEW` 管理
2. 不方便按工件类别批量遍历
3. 多实例样本、无标注样本、异常样本不容易隔离
4. 不利于 SAM3D / Cadrille 这种需要稳定目录约定的批处理流程

因此, 后续把数据统一整理成 `aiws5.2-usable/` 结构。

这个结构的核心语义是:

- 顶层按 `V1 / V2 / NEW` 分组
- 每个 subset 下再按 5 类工件分组:
  - `cover_plate` (盖板)
  - `square_tube` (方管)
  - `h_beam` (H 型钢)
  - `channel_steel` (槽钢)
  - `bellmouth` (喇叭口)
- 每个工件目录下通常包含:
  - `images/` : RGB 图像
  - `annotations/` : 对应 ISAT 标注 JSON
  - `depth_png/` 或 `depth_exr/` : 深度输入 (如果该 subset 有深度)
- 顶层辅助目录包括:
  - `metadata/` : 机器可读统计与样本清单
  - `misc/` : 不进入主实验主干的特殊样本, 内部包括:
    - `multi_instance/` : 一张图中有多个实例
    - `multi_label/` : 一张图中有多个不同类别
    - `unannotated_images/` : 有图像但无标注

当前数据状态如下:

- 当前主实验可用样本共 **1418** 个
- `V1` 共 **593** 个样本, **无深度**, 当前覆盖的工件为:
  - `cover_plate` : 200
  - `square_tube` : 99
  - `h_beam` : 100
  - `bellmouth` : 194

- `channel_steel:0`
- V2 共 **524** 个样本，全部带 **PNG** 深度，当前主实验样本均为 `cover_plate`
- NEW 共 **301** 个样本，全部带 **EXR** 深度，当前主实验样本均为 `cover_plate`
- 因此，从“工件类型实际分布”来看：
  - V1 是当前工件类型最完整的子集
  - V2 与 NEW 当前主要是带深度的 `cover_plate` 数据
  - `channel_steel` 在当前可用主视图中**暂无实例**

当前 `misc/` 中已识别的特殊情况包括：

- **23** 个 `multi_instance` 样本
- **0** 个 `multi_label` 样本
- **1** 个无标注图像样本（`NEW-G90-BLACK-24`）

当前没有把 `misc/` 数据纳入正式主实验，主要原因是这部分样本里以 **multi-instance** 标注为主，而当前 SAM3D / Cadrille 主流程默认处理的是更干净的单实例样本。若直接并入，会给标注解析、实例对应和后续批处理带来额外不确定性。

因此，`misc/` 更准确地说是**暂时未纳入当前正式口径**，并不是永久废弃。后续如果补齐多实例样本的解析、拆分或主目标选择策略，这部分数据仍然可以进一步修复并重新利用。

最终，原始平铺资源池被整理为统一的 `aiws5.2-usable/` 实验结构。

## 2.3 运行服务器（RXL）

- CPU: `2 × Intel Xeon Gold 6326 @ 2.90GHz`
- CPU 总核数: `64` 线程
- 系统内存: `503 GiB`
- GPU: `4 × NVIDIA RTX A6000`
- 单卡显存: `49140 MiB`（约 `48 GB`）
- 驱动版本: `580.82.09`

## 2.4 本轮实验中的关键硬件约束

1. **SAM3D** 主要受 **GPU 显存** 约束，但在 A6000 48 GB 上是可稳定跑通的。
2. **Cadrille PC** 模态对 GPU 调度方式敏感，若多进程错误落到同一张卡上，会触发 CUDA OOM。
3. **Cadrille IMG** 模态的首要瓶颈并不是 GPU 显存，而是 **Docker / PyTorch DataLoader** 的共享内存（**shm**）。

---

## 3. SAM3D 环境搭建结论

---

当前正式环境基线如下：

- 项目根目录： `/ssd1/rxl/zhankaiming/AIWS`
- SAM3D 仓库： `/ssd1/rxl/zhankaiming/AIWS/repos/sam-3d-objects`
- Conda 环境： `/home/rxl/anaconda3/envs/sam3d-objects`
- 当前 SAM3D 仓库基线 commit： `81a82373a3a7f4cbb00bd5b32aaf6b4d0f659ddd`

### 3.1 已补齐的关键运行资源

- `checkpoints/hf/pipeline.yaml`
- SAM3D checkpoints
- MoGe 权重
- DINOv2 本地 cache 与 checkpoint

关键路径：

- checkpoints: `/ssd1/rxl/zhankaiming/AIWS/repos/sam-3d-objects/checkpoints/hf`
- MoGe: `/ssd1/rxl/zhankaiming/AIWS/models/moge-vitl/model-real.pt`
- DINO cache: `/home/rxl/.cache/torch/hub/facebookresearch_dinov2_main`
- DINO checkpoint: `/home/rxl/.cache/torch/hub/checkpoints/dinov2_vitl14_reg4_pretrain.pth`

### 3.2 性能相关环境结论

为了让 `RTX A6000` 正常走上更高效的 attention 路径，正式运行使用：

- `ATTN_BACKEND=flash_attn`
- `SPARSE_ATTN_BACKEND=flash_attn`

实际部署中，`flash_attn` 采用了**服务器本地源码编译**，原因是预编译 wheel 与服务器 glibc/二进制环境不兼容。

更细的安装步骤与命令已整理到《环境搭建与推理 SOP》文档。

## 4. 全量实验汇总

### 4.1 SAM3D 全量实验结果

正式输出目录：

- `/ssd1/rxl/zhankaiming/AIWS/outputs/sam3d-aiws52-clean-mesh-stl-20260410-193527`

4.1.1 完成情况

| 指标   | 数值   |
|------|------|
| 总任务数 | 1418 |
| 成功数  | 1418 |
| 失败数  | 0    |
| 完成率  | 100% |

4.1.2 总体运行时间

- 最早开始时间： 2026-04-10 19:36:09
- 最晚结束时间： 2026-04-10 21:12:19
- 总墙钟时间：约 1.60 小时
- 按全量墙钟时间折算吞吐：约 884.7 样本 / 小时

分 shard 指标：

| Shard   | GPU | 完成数 | 平均单样本时延 (s) | Shard 吞吐 (样本/小时) |
|---------|-----|-----|-------------|------------------|
| shard-0 | 0   | 355 | 14.074      | 254.337          |
| shard-1 | 1   | 355 | 14.121      | 253.559          |
| shard-2 | 2   | 354 | 14.813      | 241.790          |
| shard-3 | 3   | 354 | 16.191      | 221.272          |

4.1.3 单样本时间统计（1418 样本）

| 指标               | Mean   | P50    | P90    | P95    | Max    |
|------------------|--------|--------|--------|--------|--------|
| 单样本运行时间 (s)      | 14.799 | 13.812 | 17.555 | 18.977 | 73.562 |
| 每百万像素运行时间 (s/MP) | 7.374  | 6.664  | 8.505  | 9.359  | 51.063 |

4.1.4 显存统计（1418 样本）

| 指标            | Mean      | P50       | P90       | P95       | Max       |
|---------------|-----------|-----------|-----------|-----------|-----------|
| 平均实际占用显存 (MB) | 18709.201 | 18738.090 | 19428.319 | 19614.863 | 20071.270 |
| 峰值显存上界 (MB)   | 24647.275 | 24936.000 | 26420.000 | 27136.000 | 28076.000 |

解释：

- 在 48 GB A6000 上，SAM3D 的峰值 reserved memory 上界约 **28.1 GB**，说明正式配置存在一定安全余量。
- 从显存角度看，SAM3D 在当前硬件上是可长期复用的，不需要为了全量数据额外换更高显存卡。

### 4.1.5 按数据版本与工件类型划分的运行时间与显存

为了更直接回答“不同数据版本、不同工件类型各自需要多少时间和显存”，下表按 V1 / V2 / NEW × 工件类型 给出统计。

说明：

- 时间列分别给出平均运行时间与 **P90 运行时间**
- 显存列分别给出平均实际占用显存与**峰值显存上界**
-  表示当前主实验视图中该组合没有样本

| 数据版本 | 工件类型              | 样本数 | 平均运行时间(s) | P90 运行时间(s) | 平均实际占用显存(MB) | 峰值显存上界(MB) |
|------|-------------------|-----|-----------|-------------|--------------|------------|
| V1   | cover_plate（盖板）   | 200 | 13.9323   | 16.7683     | 18695.6      | 28024.0    |
| V1   | square_tube（方管）   | 99  | 16.3360   | 17.2780     | 19110.4      | 27336.0    |
| V1   | h_beam（H 型钢）      | 100 | 11.9246   | 13.2744     | 18131.8      | 24856.0    |
| V1   | channel_steel（槽钢） | 0   | —         | —           | —            | —          |
| V1   | bellmouth（喇叭口）    | 194 | 17.8334   | 34.9408     | 18297.3      | 26298.0    |
| V2   | cover_plate（盖板）   | 524 | 14.6590   | 17.4729     | 18899.0      | 28044.0    |
| V2   | square_tube（方管）   | 0   | —         | —           | —            | —          |
| V2   | h_beam（H 型钢）      | 0   | —         | —           | —            | —          |
| V2   | channel_steel（槽钢） | 0   | —         | —           | —            | —          |
| V2   | bellmouth（喇叭口）    | 0   | —         | —           | —            | —          |
| NEW  | cover_plate（盖板）   | 301 | 14.1106   | 17.9950     | 18713.1      | 28076.0    |
| NEW  | square_tube（方管）   | 0   | —         | —           | —            | —          |
| NEW  | h_beam（H 型钢）      | 0   | —         | —           | —            | —          |
| NEW  | channel_steel（槽钢） | 0   | —         | —           | —            | —          |
| NEW  | bellmouth（喇叭口）    | 0   | —         | —           | —            | —          |

从这张表可以直接看出：

1. 当前最慢的子域是 `V1 / bellmouth`，平均运行时间约 `17.83 s`，P90 达到 `34.94 s`。
2. `V1 / square_tube` 也是相对偏慢的子域，但长尾程度明显低于 `bellmouth`。
3. `V1 / h_beam` 是当前 `V1` 中最快的一类。
4. `V2` 与 `NEW` 当前都主要体现为带深度的 `cover_plate` 负载，平均运行时间约在 `14.1–14.7 s` 区间。
5. 从显存看，各子域总体都在当前 `48 GB A6000` 的安全范围内，真正拉开差异的主要还是时间长尾，而不是显存失控。

### 4.1.6 运行瓶颈观察

主要长尾来自 `V1 / bellmouth` 子域，说明后续如果要继续优化总时长，优先分析该类别即可。

## 4.2 Cadrille 全量实验结果

### 4.2.1 PC 模态（全量成功）

用于统计的输出来自一轮“PC 成功、IMG 失败”的全量联合实验；该顶层目录由于后续清理已移入可恢复归档，但 **PC shard** 结果本身完整可用。

统计来源目录：

- `/ssd1/rxl/zhankaiming/AIWS/.trash/outputs-cleanup-20260411-153952/cadrille-full-modalities-20260411-090531-bs64/pc`

配置：

- 模态：`pc`
- 输入：SAM3D 导出的 STL
- `n_samples=5`
- `batch_size=64`
- 4 卡并行
- 选择策略：`evaluate`

结果：

| 指标               | 数值        |
|------------------|-----------|
| 送入 Cadrille 的样本数 | 1418      |
| 成功选出最佳候选数        | 1418      |
| 选择成功率            | 100.0%    |
| 墙钟时间             | 约 80.1 分钟 |

| 指标   | 数值             |
|------|----------------|
| 折算吞吐 | 约 1061.7 样本/小时 |

补充质量指标（按 shard summary 均值汇总）：

| 指标            | 数值       |
|---------------|----------|
| 平均 IoU        | 0.021192 |
| 中位 Chamfer 距离 | 0.038003 |

### 4.2.2 IMG 模态（修复后全量成功）

最终保留的成功输出目录：

- `/ssd1/rxl/zhankaiming/AIWS/outputs/cadrille-img-only-20260411-143505-shmfix`

稳定配置：

- 模态：`img`
- `n_samples=1`
- `batch_size=32`
- `--ipc=host --shm-size=16g`
- 降低 IMG dataloader worker 压力
- 选择策略：`evaluate`

结果：

| 指标               | 数值             |
|------------------|----------------|
| 送入 Cadrille 的样本数 | 1418           |
| 成功选出最佳候选数        | 1213           |
| 选择成功率            | 85.54%         |
| 选出的 STL 数        | 1213           |
| 选出的 STEP 数       | 1211           |
| 墙钟时间             | 约 25.9 分钟      |
| 折算吞吐             | 约 3290.3 样本/小时 |

补充质量指标（按 shard summary 均值汇总）：



| 指标            | 数值       |
|---------------|----------|
| 平均 IoU        | 0.026166 |
| 中位 Chamfer 距离 | 0.047796 |

### 4.2.3 Cadrille 的内存/共享内存约束结论

PC 模态约束：

- 初次多卡全量尝试时，曾出现以下典型错误：
  - `torch.OutOfMemoryError: CUDA out of memory. Tried to allocate 20.21 GiB`
- 从日志看，当时多个进程实际挤在同一张 GPU 上，导致单卡剩余显存不足。
- 结论：**PC 模态并非绝对跑不动，而是必须严格 GPU pinning，避免多进程抢同卡。**

IMG 模态约束：

- 在 `batch_size=64` 的默认路径下，4 个 shard 均出现：
  - `ERROR: Unexpected bus error encountered in worker. This might be caused by insufficient shared memory (shm).`
  - `RuntimeError: DataLoader worker ... is killed by signal: Bus error`
  - `RuntimeError: unable to write to file </torch_...>: No space left on device (28)`
- 结论：**IMG 模态的首要约束是 Docker/PyTorch DataLoader 的共享内存，而不是 GPU 显存本身。**

### 4.2.4 为什么目前没有 Cadrille 的精确峰值显存表

当前 Cadrille 运行脚本还没有像 SAM3D 一样记录：

- `torch.cuda.max_memory_allocated()`
- `torch.cuda.max_memory_reserved()`

因此，这一轮对 Cadrille 的“内存总结”更准确地说是：

- 有真实失败证据（OOM / shm bus error）
- 有稳定修复配置
- 但还没有逐样本、逐 shard 的正式峰值显存统计表

这不影响当前结论，但如果后续要写论文或正式答辩材料，建议把这组 instrumentation 加进去。

### 4.3 全链路平均端到端推理时间（基于现有全量统计折算）

由于本轮 SAM3D、Cadrille-PC、Cadrille-IMG 并不是在完全相同配置下作为单一连续任务一次性跑完，因此这里采用现有全量成功实验的墙钟时间，折算出当前系统条件下的平均端到端时间。

这里的“平均端到端时间”定义为：

- 全量墙钟时间 ÷ 样本数
- 反映的是当前 4 卡并行条件下的实际批量吞吐等效时间
- 不是单卡串行逐样本时延

先看各阶段的墙钟折算：

- SAM3D：约 1.60 小时，折算约 4.07 s / 样本
- Cadrille-PC：约 80.1 分钟，折算约 3.39 s / 样本
- Cadrille-IMG：约 25.9 分钟，折算约 1.10 s / 样本

据此可得当前全链路的平均端到端时间：

| 链路                             | 计算方式                       | 折算平均端到端时间     |
|--------------------------------|----------------------------|---------------|
| SAM3D → Cadrille-PC            | 4.07 + 3.39                | 7.46 s / 样本   |
| SAM3D → Cadrille-IMG（按送入样本数）   | 4.07 + 1.10                | 5.17 s / 样本   |
| SAM3D → Cadrille-IMG（按成功输出样本数） | (SAM3D总墙钟 + IMG总墙钟) / 1213 | 6.04 s / 成功样本 |

解释如下：

1. 如果以“送入系统的样本”为口径，当前 IMG 链路的端到端速度更快。
2. 如果以“最终成功产出”为口径，IMG 由于当前成功率约 85.54%，其端到端优势会缩小。
3. PC 链路当前更慢，但成功率是 100%，因此更适合作为稳定性优先的正式基线。

需要说明的是，PC 与 IMG 当前并不是完全同口径的纯公平对比，因为二者采用的稳定配置不同：

- PC： n\_samples=5
- IMG： n\_samples=1

因此，上述端到端时间更适合作为当前生产配置下的运行基线，而不是作为完全隔离变量后的学术性能对比。

## 5. 主要结论

1. SAM3D 已在 1418 个正式样本上完成 100% 全量推理，在 4×RTX A6000 条件下总墙钟时间约 1.6 小时。
2. SAM3D 单样本平均耗时约 14.8 秒，峰值 reserved 显存上界约 28.1 GB，说明在 A6000 48 GB 上运行是稳定的。
3. SAM3D 输出 STL 之后，Cadrille 的 PC 模态已完成全量链路验证，1418 个输入全部得到可选中的 CAD 结果。
4. Cadrille 的 IMG 模态不是算力不够，而是共享内存配置不合适；在 batch=32 + --ipc=host + --shm-size=16g + 降低 worker 后已跑通全量成功重试。
5. 当前系统已经具备“从 AIWS 图像数据，到 SAM3D 3D 重建，再到 Cadrille CAD 生成”的可复用实验基线。

## 6. 下一阶段建议

---

### 建议 1：把 Cadrille 的峰值显存统计补齐

在 `test.py` / `evaluate.py` 外围加入显存记录逻辑，补出：

- 平均显存
- P95 显存
- 最大显存
- 分模态显存表

这样后续汇报会更完整。

### 建议 2：后续批量推理采用“分阶段、分模态”策略

推荐流程：

1. 先单独跑 SAM3D，产出 STL
2. 再单独跑 Cadrille-PC
3. 最后单独跑 Cadrille-IMG

这样可以针对不同模态采用不同 batch size 与 shm 配置，运维风险更低。

### 建议 3：把 IMG 失败样本做针对性分析

当前 IMG 最终保留率约 85.5%，下一步可以重点分析：

- 哪些类别更容易失败
- 是生成失败、转换失败，还是评估选择失败
- 是否与工件几何复杂度、视角、遮挡相关

---

## 7. 附：本次汇报引用的关键输出目录

---

### 成功保留目录

- SAM3D 正式成功输出：
  - `/ssd1/rxl/zhankaiming/AIWS/outputs/sam3d-aiws52-clean-mesh-stl-20260410-193527`
- Cadrille IMG 正式成功输出：
  - `/ssd1/rxl/zhankaiming/AIWS/outputs/cadrille-img-only-20260411-143505-shmfix`

## 归档但仍可恢复的统计目录

- Cadrille PC 全量成功统计目录：
    - `/ssd1/rxl/zhankaiming/AIWS/.trash/outputs-cleanup-20260411-153952/cadrille-full-modalities-20260411-090531-bs64/pc`
- 

## 8. 一句话总结

---

在 4×RTX A6000 条件下，SAM3D 已经具备全量稳定重建能力，Cadrille 链路也已经验证可行；真正需要精细调优的重点不再是“能不能跑”，而是“如何在不同模态下更稳、更可测地跑”。